University of Debrecen
Faculty of Engineering
Mechatronics Engineering

IoT-Based Remote Monitoring and Control of the RA-4 Robotic Arm

Mini Project Proposal

CodeAlpha Robotics & Automation Internship – Task 4

Prepared by
Mustafa Ali Mohsin Albusultan

Table of Contents

1. Introduction....2
1.1 Background....2
1.2 Problem Statement....2
1.3 Objectives....2
2. The RA-4 Robotic Arm....2
3. Proposed IoT Architecture....3
4. Communication Design....4
5. Control Software and Safety Concept....4
6. Implementation Plan and Evaluation....5
7. Limitations and Future Work....6
8. Conclusion....6
References....6

Abstract

This proposal describes how the RA-4, a four-degree-of-freedom robotic arm with a two-finger gripper that was designed and simulated in Autodesk Fusion in Task 2, can be turned into an Internet of Things (IoT) device. An ESP32 microcontroller with Wi-Fi replaces a stand-alone controller and connects the arm to an MQTT broker, and a Node-RED dashboard lets an operator send commands, run pick-and-place sequences and watch the joint states and servo current from a browser. Safety-relevant functions, such as joint limits, a communication watchdog, current monitoring and a hard-wired emergency stop, are kept on the arm itself so that they do not depend on the network. The document covers the proposed architecture, the communication design, the safety concept, an eight-week implementation plan and the planned evaluation. It is a proposal; no hardware results are reported.

1. Introduction

1.1 Background

Industrial robots have long worked as isolated machines that execute a fixed program. Industry 4.0 changes this picture: machines are expected to share data with each other and with higher-level systems, so that production can be monitored, analysed and adjusted in real time [3], [4]. The Internet of Robotic Things (IoRT) describes the combination of robots with IoT technologies such as wireless networking, cloud or edge computing and standard messaging protocols [1]. According to Simoens et al. [2], the added value of this combination comes from robots that can use information from outside their own sensors and that can be supervised and coordinated remotely.

1.2 Problem Statement

The RA-4 arm designed in Task 2 has four revolute joints and a rack-and-pinion gripper, and its pick-and-place motion was verified in a motion study in Autodesk Fusion. A physical version is being 3D printed and will be driven by hobby servo motors. With a simple stand-alone controller, the arm can only repeat a program loaded through a USB cable: it cannot be operated remotely, it does not report what it is doing, and faults such as a blocked joint remain unnoticed until somebody looks at the arm.

1.3 Objectives

The aim of this mini project is to design an IoT layer for the RA-4 with the following objectives: (1) control of individual joints, the gripper and stored sequences from a web dashboard; (2) live monitoring of the joint states and the servo supply current; (3) logging of commands, states and events for later analysis; and (4) safe behaviour when the network connection is lost, a joint is overloaded or the emergency stop is pressed.

2. The RA-4 Robotic Arm

The RA-4 is a desktop arm with a base 140 mm × 140 mm in size, an upper arm of approximately 183 mm and a forearm of approximately 155 mm. Joint J1 rotates the turret about the vertical axis, J2 and J3 form the shoulder and elbow, and J4 tilts the wrist. The two gripper fingers slide on guides and are driven by a common pinion, so that they open and close symmetrically. Table 1 lists the joints with the ranges defined in the CAD model.

Table 1. Joints of the RA-4 and their ranges in the CAD model

Joint	Function	Range in the CAD model	Actuator
J1	Base rotation (vertical axis)	−135° to +135° (to be reduced to ±90°)	180° hobby servo
J2	Shoulder pitch	−30° to +150°	180° hobby servo
J3	Elbow pitch	−150° to +30°	180° hobby servo
J4	Wrist pitch	−90° to +90°	180° hobby servo
Gripper	Two fingers driven by rack and pinion	0 to 20 mm travel per finger	180° hobby servo

The servos planned for the arm have a range of 180°. Joints J2, J3 and J4 already span exactly 180°, but the CAD limit of J1 (270°) is larger than the servo range. For the physical arm, J1 will therefore be limited to ±90°, and the same limits will be stored in the firmware described below.

It should also be noted that hobby servos control their position internally but do not report it back. The joint angles shown on the dashboard are therefore the commanded angles, not measured ones; this limitation is discussed in Section 7.

3. Proposed IoT Architecture

The proposed system is organised in four layers (Figure 1). In the physical layer are the arm with its five servos and a separate servo power supply. The edge layer contains the controller: an ESP32 microcontroller, which has an integrated Wi-Fi interface [6], a PCA9685 16-channel PWM driver that generates the servo signals [7], and an INA219 sensor that measures the current drawn by the servos [8]. Both modules are connected to the ESP32 through the I2C bus, which leaves its other pins free and moves the timing-critical PWM generation to dedicated hardware.

The network layer consists of an MQTT broker. MQTT is a lightweight publish/subscribe protocol standardised by OASIS [5], in which clients do not communicate directly but publish messages on named topics, and the broker forwards them to all subscribers. For this project, the open-source broker Eclipse Mosquitto [9] is planned to run on a PC or a Raspberry Pi in the same local network. The application layer is built with Node-RED [10], a flow-based tool that can subscribe to MQTT topics and provides ready-made dashboard elements such as sliders, buttons and charts. The operator opens the dashboard in a browser on a PC or a phone; no special software is needed.

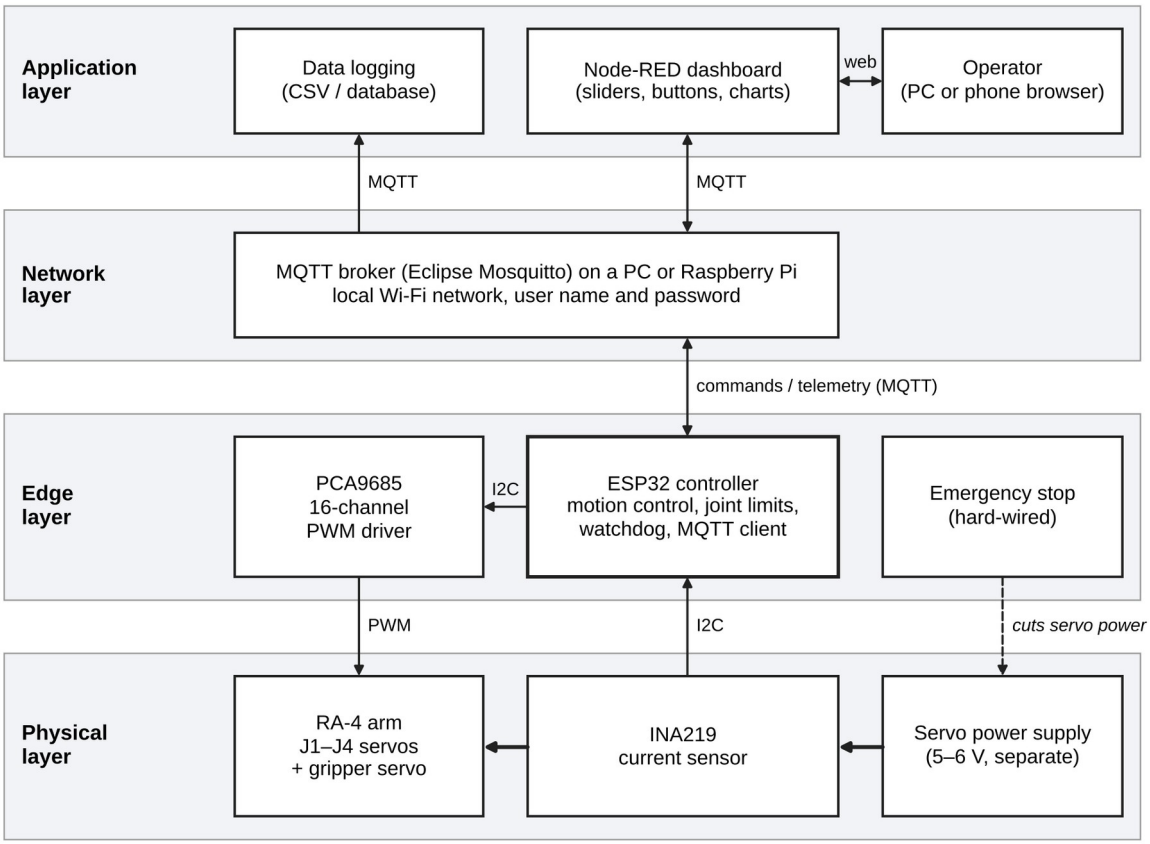


Figure 1. Proposed four-layer IoT architecture for the RA-4 arm

An important design decision is where each function runs. Everything needed to move the arm safely, namely joint limits, speed ramps, the watchdog and the current check, runs on the

ESP32. The broker and the dashboard only send requests and display data. The arm therefore never depends on the network to stay in a safe state.

4. Communication Design

All messages use a common topic prefix, ra4/, followed by the message type. Commands from the dashboard are published under ra4/cmd/, and data from the arm under ra4/state/ and ra4/event (Table 2). Payloads are short JSON objects, for example {"j": [0, 60, -90, 0], "t": 1500} for a move of all four joints to the given angles in 1500 ms. JSON is easy to create in Node-RED and easy to check on the ESP32.

Table 2. Planned MQTT topics

Topic	Direction	Content	QoS
ra4/cmd/joints	Dashboard → arm	Target angles J1–J4 and move time, JSON	1
ra4/cmd/gripper	Dashboard → arm	Open / close command	1
ra4/cmd/sequence	Dashboard → arm	Name of a stored pick-and-place sequence	1
ra4/cmd/stop	Dashboard → arm	Software stop and fault reset	1
ra4/heartbeat	Dashboard → arm	Counter sent once per second	0
ra4/state/joints	Arm → dashboard	Commanded joint angles, 5 Hz	0
ra4/state/current	Arm → dashboard	Servo supply current, 5 Hz	0
ra4/event	Arm → dashboard	Faults, rejected commands, link loss	1
ra4/status	Arm → dashboard	“online” / “offline” (retained, last will)	1

MQTT offers three quality-of-service (QoS) levels [5]. Commands and events use QoS 1, which guarantees delivery at least once; because a repeated move to the same target has no additional effect, possible duplicates are harmless. Frequent state messages use QoS 0, since a lost sample is replaced by the next one 200 ms later. The arm also registers a *last will* message with the broker: if the connection drops unexpectedly, the broker itself publishes “offline” on ra4/status, so the dashboard shows the correct status even when the arm can no longer send anything.

5. Control Software and Safety Concept

The ESP32 firmware runs a main loop every 20 ms (Figure 2). It first checks the communication watchdog: the dashboard publishes a heartbeat once per second, and if none has arrived for 3 s the arm enters a HOLD state, in which it keeps its position and rejects new motion commands until the link is restored. Next, the servo current measured by the INA219 is compared with a limit. A current well above the normal value indicates a blocked joint or a collision, so the firmware stops all motion, reports a fault and waits for a reset from the dashboard. The limit will be set during commissioning from the current measured in normal operation.

Only after these checks are new commands processed. Each command is validated: the message must have the expected format and every target angle must lie inside the joint limits of Table 1. Invalid commands are rejected and reported on ra4/event instead of being clipped silently. Valid targets are not applied at once; each joint is ramped towards its target with a maximum speed, which reduces mechanical shocks and the peak current of the servos. Finally, the joint state and the current are published at 5 Hz.

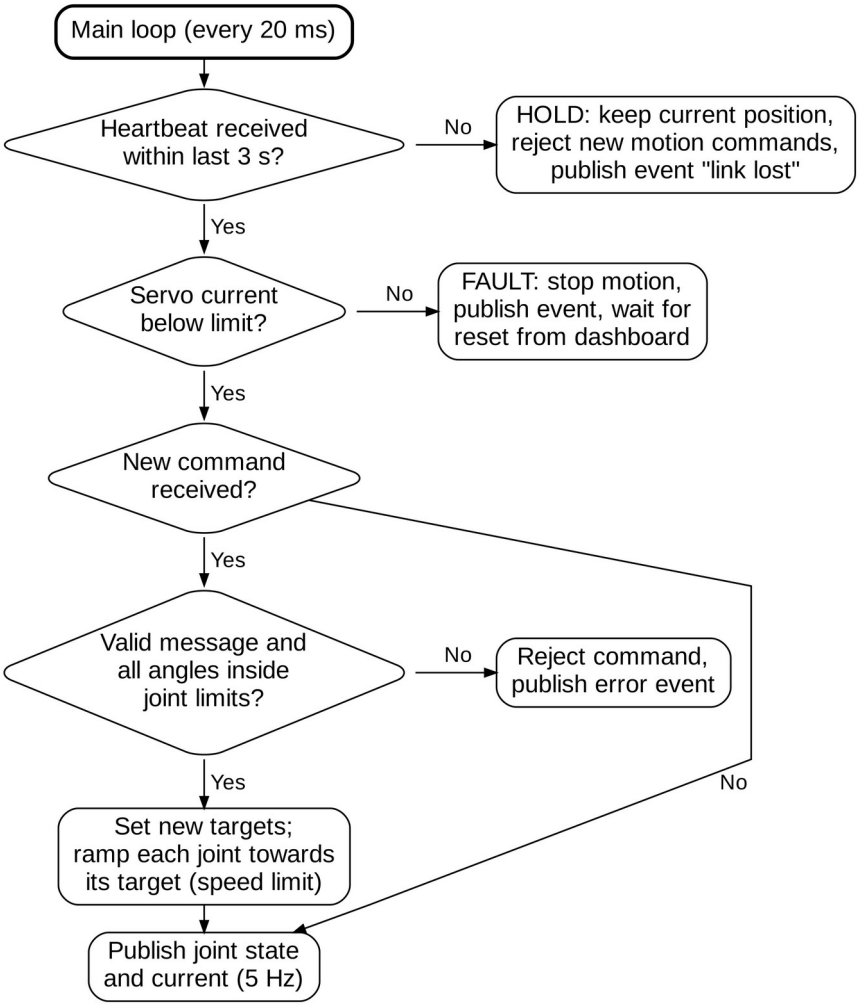


Figure 2. Main loop of the ESP32 firmware

The software functions are supported by hardware measures. A latching emergency stop interrupts the servo power supply directly, independently of the ESP32 and of the network, following the principles of ISO 13850 [11]. The servos have their own 5–6 V supply, sized for the stall current of all servos according to their data sheet, so that load peaks cannot reset the ESP32. On the network side, the broker accepts only clients with a user name and password and is reachable only from the local network. If remote access over the internet is needed later, TLS encryption should be enabled on the broker.

6. Implementation Plan and Evaluation

The work is divided into six work packages over eight weeks (Table 3). Each step ends with a test, so that problems are found while the system is still small.

The main hardware to be added to the printed arm is an ESP32 development board, a PCA9685 module, an INA219 module, a separate servo power supply and an emergency stop button. The broker and Node-RED can run on an existing PC, so no additional computer is strictly required.

Table 3. Implementation plan

Week	Work package	Result
1	Wire ESP32, PCA9685 and INA219; test each servo on the bench	Working hardware, servo pulse limits noted
2–3	Firmware: joint control, limits, speed ramps, stored sequences	Arm moves correctly from the serial console
3–4	MQTT client, topic structure, heartbeat and last will	Arm controlled through the broker
5	Node-RED dashboard and data logging	Control and monitoring from a browser
6	Watchdog, current limit and emergency stop tests	Safety functions checked one by one
7–8	Integration, pick-and-place trials, documentation	Measured results and final report

After integration, the system will be evaluated with the following measurements: (1) the delay between a command on the dashboard and the start of the movement, measured with time stamps in the log; (2) the share of successful pick-and-place cycles in a series of repeated trials; (3) the reaction to a disconnected network, checking that the arm enters HOLD within the watchdog time; and (4) the reaction to a blocked joint, checking that the current limit stops the motion. The results will be compared with the requirements in Section 1.3.

7. Limitations and Future Work

The main limitation is the missing position feedback of hobby servos. The arm can report what it was told to do, but not whether it actually reached that position. The current measurement partly compensates for this by detecting blocked joints. A later version could use servos with a feedback output or add magnetic angle sensors at the joints. Further, Wi-Fi does not give guaranteed transmission times, so the dashboard is intended for supervisory control and not for fast, real-time teleoperation. Possible extensions are a digital twin that shows the live joint states on the Fusion model, and a camera with simple image processing to locate the workpiece before each pick.

8. Conclusion

This proposal shows how the RA-4 robotic arm from Task 2 can be extended into a connected IoT system with inexpensive, widely available components. An ESP32, a PWM driver and a current sensor form the edge controller, MQTT provides a simple and standardised communication channel, and Node-RED supplies the user interface and logging. Keeping all safety-relevant functions on the arm and adding a hard-wired emergency stop ensures that the arm remains safe even when the network fails. The eight-week plan and the defined tests provide a clear path from the current CAD design to a working, measurable prototype.

References

[1] P. P. Ray, “Internet of Robotic Things: Concept, technologies, and challenges,” *IEEE Access*, vol. 4, pp. 9489–9500, 2016.

[2] P. Simoens, M. Dragone and A. Saffiotti, “The Internet of Robotic Things: A review of concept, added value and applications,” *International Journal of Advanced Robotic Systems*, vol. 15, no. 1, 2018.

[3] H. Kagermann, W. Wahlster and J. Helbig, *Recommendations for implementing the strategic initiative INDUSTRIE 4.0*. Frankfurt am Main, Germany: acatech, 2013.

[4] J. Lee, B. Bagheri and H.-A. Kao, “A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems,” *Manufacturing Letters*, vol. 3, pp. 18–23, 2015.

[5] OASIS, *MQTT Version 5.0*, OASIS Standard, 7 March 2019.

[6] Espressif Systems, *ESP32 Series Datasheet*. [Online]. Available: https://www.espressif.com

[7] NXP Semiconductors, *PCA9685: 16-channel, 12-bit PWM Fm+ I²C-bus LED controller*, product data sheet. [Online]. Available: https://www.nxp.com

[8] Texas Instruments, *INA219 Zero-Drift, Bidirectional Current/Power Monitor With I²C Interface*, data sheet. [Online]. Available: https://www.ti.com

[9] Eclipse Foundation, “Eclipse Mosquitto: An open source MQTT broker.” [Online]. Available: https://mosquitto.org

[10] OpenJS Foundation, “Node-RED documentation.” [Online]. Available: https://nodered.org/docs

[11] ISO 13850:2015, *Safety of machinery – Emergency stop function – Principles for design*. Geneva, Switzerland: International Organization for Standardization, 2015.